

PRACTICAL WEEK12

TASK1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
rishil10@ASUS16X: ~/Practica × + ▾
GNU nano 8.7.1 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 10000

int BUFFER_SIZE;
int *buffer;
int in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}
```

```
rishill10@ASUS16X: ~/Practica × + ▾
GNU nano 8.7.1 producer_consumer.c *

int main()
{
    printf("Enter buffer size: ");
    scanf("%d", &BUFFER_SIZE);

    buffer = (int *)malloc(BUFFER_SIZE * sizeof(int));

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    clock_t start = clock();

    pthread_t prod, cons;
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    clock_t end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nProducer-Consumer completed successfully.\n");
    printf("Buffer size: %d\n", BUFFER_SIZE);
    printf("Items produced: %d\n", ITEMS);
    printf("Items consumed: %d\n", ITEMS);
    printf("Execution time: %f seconds\n", time_taken);

    if (time_taken > 0)
    {
        printf("Throughput: %.2f items/second\n", ITEMS / time_taken);
    }

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    free(buffer);
    return 0;
}
```

```
rishill10@ASUS16X: ~/Practica × + ▾
rishill10@ASUS16X:~$ mkdir Practical12
cd Practical12
rishill10@ASUS16X:~/Practical12$ nano producer_consumer.c
rishill10@ASUS16X:~/Practical12$ gcc producer_consumer.c -o producer_consume
r -pthread
./producer_consumer
Enter buffer size: 5

Producer-Consumer completed successfully.
Buffer size: 5
Items produced: 10000
Items consumed: 10000
Execution time: 0.065306 seconds
Throughput: 153125.29 items/second
rishill10@ASUS16X:~/Practical12$ |
```

TASK2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
GNU nano 8.7.1      deadlock.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Locked Resource 1\n");
    sleep(1);

    printf("Thread 1: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    return NULL;
}

void *thread2_function(void *arg)
{
    printf("Thread 2: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Locked Resource 2\n");
    sleep(1);

    printf("Thread 2: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Locked Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);
    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed.\n");
    return 0;
}
```

```
rishil10@ASUS16X: ~/Practica × + ▾
GNU nano 8.7.1 deadlock_prevention.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Locked Resource 1\n");
    sleep(1);

    printf("Thread 1: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Locked Resource 2\n");

    printf("Thread 1: Using both resources\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    printf("Thread 1: Resources released\n");
    return NULL;
}

void *thread2_function(void *arg)
{
    printf("Thread 2: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Locked Resource 1\n");
    sleep(1);

    printf("Thread 2: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Locked Resource 2\n");

    printf("Thread 2: Using both resources\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    printf("Thread 2: Resources released\n");
    return NULL;
}

return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed successfully.\n");
    return 0;
}
```

```
rishill10@ASUS16X: ~/Practica × + ▾
rishill10@ASUS16X:~/Practical12$ nano deadlock.c
rishill10@ASUS16X:~/Practical12$ gcc deadlock.c -o deadlock -pthread
./deadlock
Thread 1: Trying to lock Resource 1
Thread 1: Locked Resource 1
Thread 2: Trying to lock Resource 2
Thread 2: Locked Resource 2
Thread 2: Trying to lock Resource 1
Thread 1: Trying to lock Resource 2
^C
rishill10@ASUS16X:~/Practical12$ nano deadlock_prevention.c
rishill10@ASUS16X:~/Practical12$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
./deadlock_prevention
Thread 1: Trying to lock Resource 1
Thread 1: Locked Resource 1
Thread 2: Trying to lock Resource 1
Thread 1: Trying to lock Resource 2
Thread 1: Locked Resource 2
Thread 1: Using both resources
Thread 1: Resources released
Thread 2: Locked Resource 1
Thread 2: Trying to lock Resource 2
Thread 2: Locked Resource 2
Thread 2: Using both resources
Thread 2: Resources released
Program completed successfully.
rishill10@ASUS16X:~/Practical12$ |
```